

Campus Tasks: State That Survives.

Advanced state management and local Room persistence—
so the app remembers what matters.

— From a working screen to a dependable application.

01

State

Make the screen's current truth explicit.

02

Room

Store structured data on the device.

03

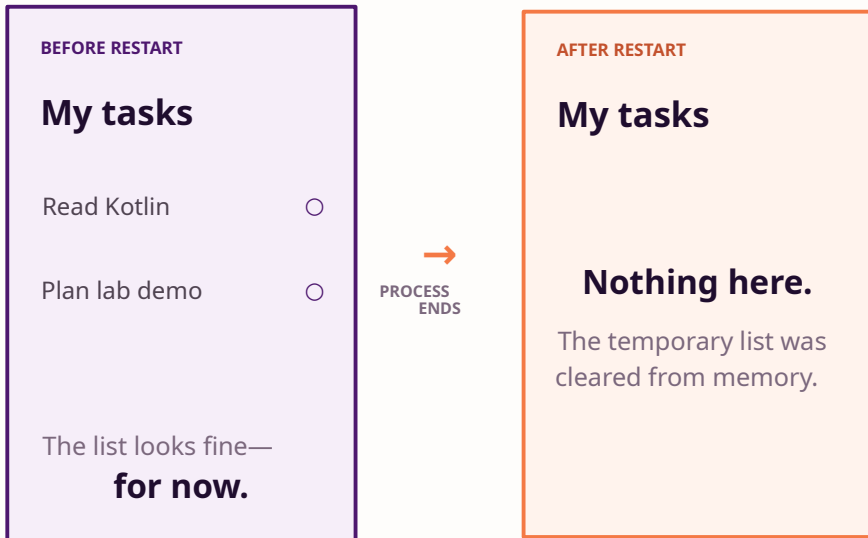
Proof

Restart the app. The work remains.

A task should not vanish because the app took a break.

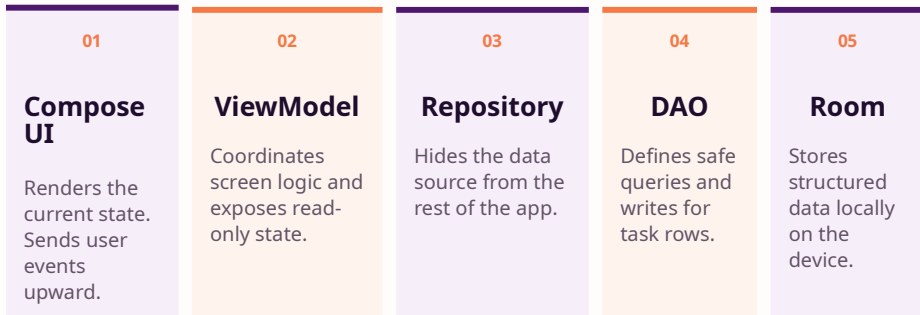
Our first version stored tasks in a Kotlin list. That is useful for practice—but the list lives only while the process is alive.

Ask the room: What should survive a restart—a text-field draft, a saved task, or a loading spinner?



One reliable data path.

Each layer has one job. When the jobs are separated, the app becomes easier to explain, test, and change.



EVENTS UP ↑ STATE DOWN ↓



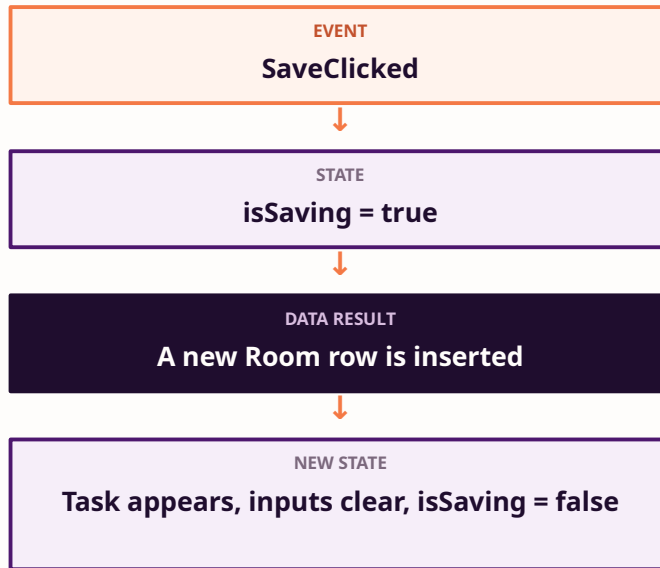
State is; events happen.

State describes the screen right now. Events are transient inputs that cause the state to change.

A single UI State

A single `TaskListUiState` object keeps all possible screen conditions visible and predictable:

- **Loading:** show progress while querying
- **Empty:** explain what to do when there are no tasks
- **Content:** show tasks with clear completion states
- **Validation:** explain why a title cannot be saved
- **Error:** give a readable message and a retry path



Room has three essential parts.

01

Entity

The shape of a table row. It defines the exact data structure that will be stored in the local database.

02

DAO

The safe doorway for queries and writes. It contains the SQL operations needed to access the entities.

03

Database

The application's access point to Room and its DAOs. It ties the entities and DAOs together into a single source of truth.

Room adds compile-time SQL checking, annotations, and migration support on top of standard SQLite.

Step 1 — Model a task

An **Entity** defines the exact shape of a table row in the local database.

THINK LIKE A DESIGNER

Which fields are required? Which belong to the database but not the screen?

TASKENTITY.KT

```
@Entity (tableName = "tasks")
data class TaskEntity
(
    @PrimaryKey (autoGenerate = true
,    val id: Long = 0

,    val title: String
,    val note: String = ""
,    val isCompleted: Boolean = false
,    val priority: Int = 2
,    val createdAt: Long = System
    .currentTimeMillis()
)
```

Step 2 — Observe and change rows

Flow for observation

A **Flow** lets the UI receive continuous updates whenever the database changes. No manual refreshing required.

Suspend for writes

Writes are **suspend** functions because they take time and should never block the main UI thread.

```
@Dao interface TaskDao { @Query("SELECT * FROM tasks  
ORDER BY isCompleted ASC, createdAt DESC") fun  
observeTasks(): Flow<List<TaskEntity>> @Insert suspend  
fun insert(task: TaskEntity): Long @Update suspend fun  
update(task: TaskEntity) @Delete suspend fun  
delete(task: TaskEntity) }
```

Step 3 — One database, one repository

APPDATABASE.KT

```
@Database(entities = [TaskEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun taskDao(): TaskDao
}
```

TASKREPOSITORY.KT

```
class TaskRepository(private val dao: TaskDao) {
    val tasks = dao.observeTasks()

    suspend fun add(task: TaskEntity) = dao.insert(task)
}
```

Build it once

Construct the database once for the entire application process.

Pass the DAO

The repository takes the DAO as a dependency, acting as the gateway.

Hide the details

Keep Room annotations and database logic completely out of the Compose screen.

Step 4 — Let the ViewModel own screen state

The **ViewModel** survives screen rotations. It collects data from the repository and exposes it to the UI as an immutable **StateFlow**.

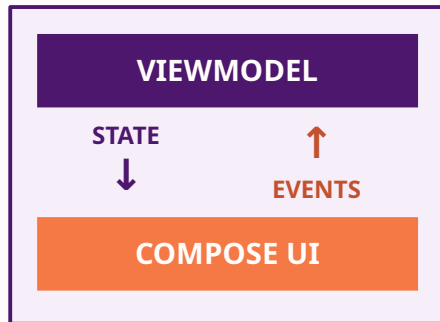
TASKVIEWMODEL.KT

```
class TaskViewModel ( private val repository: TaskRepository ) : ViewModel() {  
    > val uiState: StateFlow< TaskListUiState> =  
        repository.tasks  
            .ContentAsStateFlow() TaskListUiState  
            .LoadingStateIn(scope = viewModelScope, started = SharingStarted.WhileSubscribed( 5000 ), initialValue =  
                TaskListUiState.Loading )  
    fun addTask (title: String)  
    { viewModelScope.launch { repository.add( TaskEntity(title) ) } }  
}
```

Step 5 — Events move up; state flows down

TASKSCREEN.KT

```
@Composable
fun TaskScreen (viewModel: TaskViewModel) {
    viewModel.state.collectAsStateWithLifecycle()
    ( TaskList
        tasks = state.tasks,
        onSave = { title -> viewModel.saveTask(title) },
        onToggle = { task -> viewModel.toggleTask(task) }
    )
}
```



The Golden Rule

Never pass the **ViewModel** down to child components. Pass only the **data** they need to display, and the **functions** they need to call.

Design for every database moment.

A reliable app handles more than just the happy path. Ensure your UI accounts for all possible states.

Loading

Show a clear progress indicator while Room queries the disk or performs initial setup.

Empty

Guide the user with helpful placeholder text when the database has no tasks to display.

Content

Display tasks clearly with touch-friendly rows (minimum 48dp height) and legible typography.

Validation

Prevent empty or invalid data from reaching Room. Disable the save button until input is valid.

Error

Provide a readable message and a clear way to retry if a database operation fails.

Accessibility

Ensure screen readers announce state changes, like "Task saved" or "Loading complete."

Campus Tasks: **The Persistent App**

Replace the temp list with a local database. Build the data path from Room to the Compose UI.

01 Deliverables

- **Data:** Task Entity, DAO, Room DB.
- **Domain:** TaskRepository interface.
- **UI:** ViewModel exposing StateFlow.
- **Compose:** UI reacts to state only.

02 Success Criteria

- **Persistence:** Tasks survive restarts.
- **State:** Empty/loading/error visible.
- **Validation:** No empty tasks saved.
- **Actions:** Toggle and delete work.

03 Extension Challenge

- ★ **Priority:** Add High/Med/Low and sort.
- ★ **Due Date:** Add timestamp field.
- ★ **Search:** Filter via DAO query.